Trabajo Práctico Integrador — Segunda Entrega (Unidades 4 a 8)

Food Store: Cierre Integral de la Capa de Datos (PostgreSQL 18 · MongoDB · SQLite 3.50+)
Estudiante: Fabrizio Simon · **Cátedra:** Base de Datos II (UTN FRM) · **Ciclo Lectivo:** 2026

Repositorios Oficiales en GitHub:
• **Parte 1 (Unidades 1 a 6):** https://github.com/simonfabrizio103/food-store-tp (Carpetas facultad/food-store-tp-unidad4/, u5/, u6/ en rama master, commit 202aecb).
• **Parte 2 (Unidades 7 y 8):** https://github.com/simonfabrizio103/food-store-tp-u7-u8 (Carpetas tp-unidad-7/ y tp-unidad-8/ en rama main, etiqueta oficial **tp8**).

Introducción y Enfoque Personal: Para esta segunda entrega del TPI integré las unidades 4 a 8 cubriendo desde la refactorización avanzada en PostgreSQL hasta la arquitectura políglota con MongoDB y la base embebida Offline-First en SQLite para las tablets del Food Truck. En cada etapa trabajé sobre copias de prueba (food_store_copia, foodstore_restore_test), medí tiempos reales con EXPLAIN y cronómetros de PowerShell, y mantuve una auditoría técnica estricta sobre las sugerencias de la IA, descartando propuestas inviables para producción.

1. Qué elementos implementé en cada unidad (4 a 8)

Unidad / Eje Temático	Componentes y Scripts Implementados	Descripción Funcional y Arquitectónica
Unidad 4 **Normalización avanzada y redundancia**	• `tp_fnbc_control_lote.sql` • `tp_desnormalizacion_top_categorias.sql` • `Vista de compatibilidad y triggers`	Descompuse control_lote_almacen a FNBC eliminando la dependencia ResponsableControlID → DepositoID sin pérdida de datos. Creé la tabla agregada categoria_venta_dia (top 5 categorías diarias) con triggers de sincronización y auditoría periódica.
Unidad 5 **Administración y seguridad**	• `sql/roles.sql (RBAC)` • `sql/auditoria_test.sql` • `sql/simulacro_incidente.sql` • `sql/usuario_anon.sql`	Diseñé el esquema RBAC de mínimo privilegio con roles de grupo y login, permisos a nivel de columna sobre usuario.email y default privileges. Activé auditoría nativa (log_connections, log_disconnections, log_statement = all), ejecuté simulacro forense y apliqué pseudonimización determinista con sal.
Unidad 6 **Mantenimiento de bases de datos**	• `maintenance.sql` • `backup_foodstore.ps1` • `restore_test.ps1` • `validate_restore.sql`	Formulé el plan de mantenimiento preventivo (VACUUM ANALYZE, REINDEX CONCURRENTLY, estadísticas). Automaticé backups diarios lógicos comprimidos con rotación de 7 días. Ejecuté simulacro de recuperación contra reloj en base efímera y validé integridad.
Unidad 7 **NoSQL (MongoDB) y políglota**	• `consultas_mongodb.js` • `consultas_postgresql.sql` • `ejecutar_mongodb.js` • `SPEC_U7.md / BITACORA.md`	Modelé la colección pedidos embebiendo líneas con precio unitario congelado e instantánea del cliente. Desarrollé 4 consultas analíticas en MongoDB y en PostgreSQL con 100% de coincidencia exacta, fundamentando la arquitectura híbrida CQRS.
Unidad 8 **Bases de datos móviles e integración**	• `schema_sqlite.sql (STRICT)` • `semaforo.sql / simulador.py` • `pruebas_sqlite.sql` • `servidor_snapshot.csv`	Implementé la base SQLite para la tablet del Food Truck en modo Offline-First con tablas STRICT, precios en centavos enteros y UUIDs v4. Creé el algoritmo de semáforo de conciliación (Rojo/Amarillo/Verde), simulé casos C1 a C7 y comprobé la regla de oro en PostgreSQL.

2. Cómo probé el funcionamiento

Técnica de Prueba	Unidad / Escenario Real Probado	Metodología y Comprobación en el Motor
Transacciones reversibles (BEGIN / ROLLBACK)	Unidad 4 (Migración FNBC) Unidad 8 (Borradores móviles)	En U4 ejecuté la desestructuración de lotes dentro de bloques de transacción para validar que no hubiera tuplas huérfanas antes de confirmar. En U8 probé la anulación limpia de un pedido en mostrador con ROLLBACK, comprobando mediante COUNT(*) = 0 que no quedó basura.
Copias y bases réplica de prueba	Unidades 4, 5 y 6 food_store_copia y restore_test	Toda modificación de esquema y simulacro destructivo se realizó sobre bases aisladas con datos sintéticos volumétricos (200,001 pedidos y 50,000 productos), resguardando los ambientes de desarrollo limpios y la base productiva.
Simulacro de acceso indebido	Unidad 5 (Seguridad) Rol atacante: app_web	Ejecuté simulacro_incidente.sql forzando ataques desde app_web: lectura masiva, intento de cambio a admin_datos, creación de usuarios espurios y llamadas a fn_autenticar. Verifiqué los bloqueos en el log y el informe forense.
Simulacro de desastre contra reloj	Unidad 6 (Continuidad) Script: restore_test.ps1	Automaticé la restauración cronometrada desde el último backup comprimido hacia foodstore_restore_test. Validé la integridad estructural con validate_restore.sql chequeando claves foráneas, índices y cantidad de registros.
Casos de conflicto de sincronización	Unidad 8 (Móvil) Semáforo de conciliación (C1 a C7)	Probé metódicamente los casos de la feria: consulta offline (C1), subida de precio y aceptación (C2 → Conflicto → Aprobado), quiebre de stock (C3 → Rechazo), baja lógica/lápida (C4 → Rechazo y ocultamiento de catálogo), idempotencia (C6) y restricciones estrictas (C7).

3. Qué resultados obtuve

A continuación se detallan las mediciones reales, métricas de recuperación y resultados empíricos obtenidos en las pruebas de base de datos:

Métrica / Escenario Evaluado	Resultado Numérico y Comportamiento Observado	Evidencia e Impacto en el Proyecto
Rendimiento Desnormalización (U4)	Consulta original: **0.123 ms** (Seq Scan) Con tabla desnormalizada: **0.077 ms** (Bitmap Heap Scan)	**37.4% de reducción en tiempo de respuesta** en el cálculo del Top 5 de categorías. Auditoría periódica reportó **0 tuplas desincronizadas**.
Hallazgo Crítico Forense H-1 (U5)	Fallo de autenticación por SECURITY INVOKER. Remediación: cambio a SECURITY DEFINER.	app_web no podía validar login al carecer de SELECT sobre usuario. Se securizó la función bajo el owner admin_datos con search_path fijo.
Simulacro de Desastre: RTO (U6)	**RTO medido: 4.84 segundos** (Restauración completa de base de 96 MB y 200,001 órdenes)	Superó con holgura el objetivo fijado de RTO ≤ 15 min. La base réplica quedó disponible y validada con 0 inconsistencias de foreign keys.
Simulacro de Desastre: RPO (U6)	**RPO medido: 1.70 minutos** (Tiempo entre último respaldo/WAL y el incidente)	Muy por debajo del límite máximo tolerable de 1 hora fijado para el negocio de Food Store en el informe post-incidente.
Paridad Relacional vs Documental (U7)	Consulta 4.a (Entregados): 3 pedidos (IDs 2, 4, 7) Consulta 4.b (Prod. 3): 3 pedidos Consulta 4.c (Total > $2000): 2 pedidos (IDs 4 y 6)	**100% de coincidencia exacta** en identificadores, cantidades y montos entre MongoDB y PostgreSQL, validando la integridad del modelo documental.
Invariabilidad del Servidor Central (U8)	Conteo en PostgreSQL tras semáforo: **200,001 pedidos**. Stock de productos 1, 2, 3 y 4: **5, 10, 115 y 130**.	**Cero pedidos agregados y cero stock descontado en PostgreSQL.** Se validó empíricamente que SIMULADO_APROBADO no es una venta.

4. Qué decisiones de diseño tomé

A lo largo del proyecto prioricé soluciones de ingeniería que balancean consistencia, seguridad, rendimiento y desacoplamiento:

Área de Diseño	Decisión de Arquitectura Adoptada	Justificación Técnica y Salvaguardas Implementadas
Refactorización FNBC vs Compatibilidad (U4)	Separación en dos tablas normalizadas con **vista de compatibilidad**.	Al extraer responsable_control se eliminaron anomalías de actualización. Para no romper aplicaciones preexistentes, creé una vista con el nombre y columnas de la tabla original, haciendo la transición 100% transparente.
Criterio de Desnormalización (U4)	Desnormalización selectiva de categoria_venta_dia con triggers.	Solo desnormalicé la consulta crítica del dashboard para aliviar la CPU en horas pico. Como salvaguarda, implementé triggers para actualización automática y un script de auditoría que alerta cualquier discrepancia.
Mínimo Privilegio y Seguridad Funcional (U5)	Cierre de permisos directos y uso de SECURITY DEFINER seguro.	El rol app_web no tiene SELECT sobre la tabla usuario ni acceso a contraseñas. La validación de credenciales se encapsuló en fn_autenticar con owner admin_datos y search_path = public, pg_temp.
Patrón Híbrido Políglota / CQRS (U7)	PostgreSQL como núcleo OLTP y MongoDB como Read Model desnormalizado.	No forcé una migración excluyente. PostgreSQL es la única fuente de verdad (ACID, inventario, auditoría y facturación); MongoDB recibe eventos de ventas confirmadas para servir lecturas web masivas a muy baja latencia.
Diseño Móvil Offline-First (U8)	SQLite en modo STRICT, precios en centavos enteros y UUIDs v4.	Eliminé errores de redondeo monetario con centavos enteros. Usé UUIDs v4 para que múltiples tablets en la feria no colisionen IDs de pedidos. Rige la regla de oro: *el móvil propone borradores, pero solo PostgreSQL confirma ventas*.

5. Herramientas de IA utilizadas, bitácora y descartes fundamentales

Utilicé Kiro para el diseño formal de requisitos EARS y especificaciones técnicas, y asistentes de código (OpenCode y LLMs) para prototipado rápido. Sin embargo, mantuve un criterio estricto de validación humana: **toda propuesta fue auditada, probada en el motor y refutada cuando violaba buenas prácticas**:

ID / Unidad	Componente	Propuesta de la IA	Decisión Humana	Fundamentación Técnica y Motivo de Rechazo
CR-1 Unidad 6	maintenance.sql	Ejecutar VACUUM FULL ANALYZE diariamente para compactar espacio.	**DESCARTADO DE PLANO**	VACUUM FULL solicita bloqueo exclusivo (ACCESS EXCLUSIVE LOCK), impidiendo lecturas y escrituras durante minutos. En una base con 200,000 pedidos, tiraría la tienda abajo. Mantuve VACUUM (ANALYZE, VERBOSE) concurrente.
CR-2 Unidad 8	schema_sqlite.sql	Usar claves primarias autonuméricas INTEGER AUTOINCREMENT en pedidos locales.	**DESCARTADO DE PLANO**	En un entorno distribuido offline con múltiples tablets, los enteros autoincrementales colisionan inevitablemente al sincronizar en el servidor central. Se impuso **UUID v4**.
CR-3 Unidad 8	Triggers en SQLite	Crear triggers locales que intenten actualizar stock remoto en PostgreSQL por socket.	**DESCARTADO DE PLANO**	Viola la regla de oro y la premisa Offline-First: si la tablet no tiene señal en la feria, la interfaz se colgaría esperando el timeout de red. La conciliación debe ser asíncrona mediante el semáforo.
CR-4 Unidad 7	Modelado NoSQL	Afirmar que 'MongoDB carece de soporte para transacciones ACID'.	**REFUTADO TÉCNICAMENTE**	MongoDB soporta transacciones ACID multi-documento desde la versión 4.0 (2018), y además en agregados con líneas embebidas la atomicidad es nativa a nivel de documento sin overhead de coordinación.
CR-5 Unidad 5	fn_autenticar	Conceder permisos directos de SELECT a app_web sobre usuario.	**DESCARTADO**	Viola el mínimo privilegio y expone columnas confidenciales (hashes de contraseñas). La solución profesional fue convertir la función a SECURITY DEFINER.

Declaración de Uso de IA (DUIA): Todas las interacciones, especificaciones en Kiro, prompts empleados y fundamentaciones de descarte fueron debidamente asentadas en las bitácoras (BITACORA.md) y declaraciones juradas (DUIA.md) de ambos repositorios, acreditando el dominio conceptual y técnico del estudiante para defender cada línea de código en la instancia oral.

Conclusión Final

Esta segunda entrega me demostró la importancia de concebir la base de datos como una arquitectura integral y especializada. Aprendí que optimizar no es agregar índices sin medir sino estudiar el plan del optimizador; que la seguridad radica en el mínimo privilegio real y funciones securizadas; que un plan de contingencia requiere pruebas cronometradas de restauración (RTO de 4.84 s y RPO de 1.70 min); y que en arquitecturas modernas la complementariedad entre PostgreSQL (núcleo transaccional), MongoDB (read-models desnormalizados) y SQLite (borde desconectado) permite construir soluciones empresariales de alta disponibilidad, seguras y escalables.